



CONFIDENTIAL — ARCHITECTURE

Quenyx AI Platform Bible

Quenyx vOPS HUB — v1.0.0 RC1 · Document v2.0

Document Version	2.0
Software Version	v1.0.0 RC1
Applies To	Quenyx vOPS HUB v1.0.0 RC1
Classification	Confidential — Architecture
Owner	AI Platform Engineering
Status	Released
Last Updated	2026-06-29
Document Type	Architecture reference

REVISION HISTORY

Version	Date	Notes
1.0	2026	Initial AI platform bible (through Sprint 19).
2.0	2026-06-29	RC1 alignment: Quenyx AI documented as a shared platform layer (not "QynShield AI"); explicit future-adaptor list; Unified AI Workspace (Sprint 20).

Table of Contents

- 1. AI Platform vision
- 2. Provider abstraction
- 3. OpenAI provider ●
- 4. Mock provider ●
- 5. Provider registry
- 6. Skill registry
- 7. Skill router
- 8. Module adapters
- 9. QynShield adapter ●
- 10. QynSight reserved adapter ●
- 11. AI contract (request DTO)
- 12. Reasoning engine ●
- 13. Retrieval foundation ● / ●
- 14. RAG foundation ●
- 15. Copilot ● / ●
- 16. Demo mode ●
- 17. Guardrails
- 18. Prompt logging policy
- 19. Conversation persistence policy
- 20. Future module adapters ● / ●
- 21. No direct DB from AI core
- 22. No direct provider calls outside providers
- 23. Capability catalog ●
- Quenyx AI — Unified AI Workspace (Sprint 20)

07 — Quenyx AI Platform Bible

Audience: Architects, AI engineers. **Scope:** The shared, platform-wide Quenyx AI Platform (extracted in Sprint 19, surfaced platform-wide by the Unified AI Workspace in Sprint 20), plus the QCIF AI runtime (Sprints 9–18) it consumes.

1. AI Platform vision

Quenyx AI is a **shared, governable AI runtime for the whole vOPS HUB** — it is not "QynShield AI" and is not a feature of any single module. Modules plug in through a thin **adapter**; the platform owns provider abstraction, skill routing, prompt orchestration, retrieval/RAG, and the capability catalog. The platform is **deterministic-first**: business logic decides *what*; AI only *renders*.

Current platform-wide capabilities: **AI Provider Abstraction, AI Skills, AI Context, Retrieval, Reasoning, Knowledge Graph, Gap Engine, Evidence Engine, Recommendation Engine, Copilot, RAG Runtime, Executive Platform**, provider abstraction, and **workspace isolation**.

2. Provider abstraction

All model access goes through a **provider interface** resolved by `AiProviderRegistry`. **No model is hardcoded** — names come from env (`OPENAI_MODEL` , `OPENAI_EMBEDDINGS_MODEL`). No code outside a provider class makes a direct model/HTTP call.

3. OpenAI provider

`OpenAiProvider` implements the provider interface for OpenAI (chat + embeddings). Active only when `AI_PROVIDER=openai` and `AI_ENABLED=true`; otherwise it is never invoked.

4. Mock provider

`MockAiProvider` is the **default** (`ai.default = mock`). It returns deterministic mock responses so the entire platform runs with **zero external calls** — ideal for demos, tests, and safe defaults.

5. Provider registry

`AiProviderRegistry` maps a provider key → implementing class from `config/ai.php`. Selecting an unimplemented future provider (azure/claude/gemini/ollama/local — declared, not built) throws.

6. Skill registry

`config/ai.skills.registered` maps skill keys to classes with priorities and per-skill flags:
`corpus_search` , `knowledge_graph` , `framework_mapping` , `evidence` , `gap_assessment` ,

`recommendation` . **Skills make no AI calls** — they reuse the deterministic compliance services and return AI-context payloads.

7. Skill router

The orchestrator auto-routes a request to skills by priority (higher first) and supported context. Skills are individually feature-flagged and can be disabled via env.

8. Module adapters

`AIModuleAdapterInterface` defines the contract every module implements:

```
moduleKey(): string
supportedSkills(): array
supportedContexts(): array
buildContext(AIModuleRequest $request): array
buildReasoning(AIModuleRequest $request, array $context): ReasoningOutput
buildPrompt(AIModuleRequest $request, ReasoningOutput $reasoning): AiPrompt
```

The 3-stage pipeline (**context** → **reasoning** → **prompt**) keeps deterministic work separate from model rendering.

9. QynShield adapter

`QynShieldAiAdapter` is the **first live adapter**. It wraps existing QynShield services (planner, retrieval, reasoning, prompt orchestrator, skills registry) — **no business logic was duplicated or moved**. Registered at boot with `QuenyxAiPlatform::registerAdapter( ... )`.

10. QynSight reserved adapter

`QynSightAiAdapterInterface` exists as a **reserved contract only**. There is **no QynSight AI implementation** — no monitoring AI, RCA, incident AI, log analysis, or metrics AI. When built, its adapter registers with **no platform change** and its `module_catalog` status flips `reserved` → `production` .

11. AI contract (request DTO)

`AIModuleRequest` is the generic, module-agnostic request DTO (module key, skill/context, payload). This is the single shape the platform understands, regardless of module.

12. Reasoning engine

The **Compliance Reasoning Engine** (Sprint 16) is deterministic and rule-based (`ComplianceReasoningRuleSet`). It decides what to answer **before** any AI, producing a

`ReasoningOutput` (findings, applied rule IDs, explanation). Disabling it falls back to legacy skill-composed prompts. It makes **no AI or DB calls** itself.

13. Retrieval foundation /

`ComplianceRetrievalService` provides deterministic retrieval over the corpus (Sprint 15). It exposes `queryDetailed()` for downstream RAG. Retrieval itself is deterministic; RAG adds optional vector recall on top.

14. RAG foundation

Sprint 17 hybrid RAG:

- `ComplianceHybridRetrievalService` merges deterministic + optional vector results, de-duplicates, and ranks; **falls back to deterministic** if a vector provider fails.
- `ComplianceRagContextBuilder` builds a **bounded, cited** context package (token budget, citations, guardrails). It makes **no AI call**.
- `OpenAiVectorRetrievalProvider` runs **metadata-only** (no fake similarity); off by default.
- **Tenant evidence is never indexed by default** (`rag.index_tenant_evidence = false`).

15. Copilot /

`ComplianceCopilotService` orchestrates: scope resolve → retrieval → deterministic reasoning → (optional RAG context) → prompt orchestration → provider. **Citation-enforced** ("no source, no answer"). Runs in **mock mode** unless `AI_ENABLED=true` .

16. Demo mode

`AI_COPILOT_DEMO_MODE` adds a `demo` block exposing **existing** intelligence — reasoning trace, fired rules, citations, retrieved chunks, recommendation sources, evidence chain. It is **not chain-of-thought** and adds **no new reasoning**. Off by default.

17. Guardrails

- AI off by default; mock provider default.
- Deterministic reasoning gates the answer.
- Citations enforced; uncited chunks excluded.
- Token budget bounds context.
- No tenant-evidence embeddings by default.
- Provider-agnostic; no hardcoded models.

18. Prompt logging policy

`prompt_logging` (and Copilot's `AI_PROMPT_LOGGING_ENABLED`) default **false** — user/assistant prompt **content is never written to storage** unless explicitly enabled.

19. Conversation persistence policy

`persist_conversations` (and `AI_CONVERSATION_PERSISTENCE_ENABLED`) default **false** — Copilot can operate without storing any message content.

20. Future module adapters

Each HUB module can become an AI consumer by implementing `aiModuleAdapterInterface` and registering — inheriting providers, skills, prompt orchestration, retrieval/RAG, reasoning, and the capability catalog automatically. **Future AI adapters** are planned for: **QynSight, QynAsset, QynRun, QynNotify, QynReact, QynKnow, QynSupport, QynBalance, and QynVA**. (`QynCore` is the platform core, not an adapter target; there is no `QynIntegrations` module.)

21. No direct DB from AI core

The AI core does not touch the database directly for business data; it consumes **deterministic service outputs** (via skills/adapters). DB access lives in the compliance/services layer, not in the AI orchestration core.

22. No direct provider calls outside providers

Reaffirmed and **verified by static scan**: only provider classes (and the metadata-only vector provider, via the registry) reference the model SDK. (The legacy QynSight knowledge-base agent `Services/OpenAI/OpenAIService` predates the platform and is a separate, isolated service.)

23. Capability catalog

`GET /api/ai/platform/capabilities` (`QuenyxAiCapabilityCatalog`) dynamically returns registered **modules** (adapters), **skills, providers, reasoning/retrieval/RAG** status, supported contexts, and the HUB-wide `module_catalog` (production/reserved/planned), independent of UI visibility.

Quenyx AI — Unified AI Workspace (Sprint 20)

RC1.1: surfaced in the UI as **Quenyx AI** (the enterprise AI control center). Internal/codename remains *Unified AI Workspace*; API (`/api/ai/*`) and SPA (`/ai-workspace/*`) paths are unchanged and a branded `/quenyx-ai/*` alias redirects to them.

A platform-level surface that exposes the shared AI runtime to every workspace through flat `/api/ai/*` endpoints (see API Reference §17). It **reuses** the existing runtime rather than adding new AI logic:

- **Conversations / Chat / History** reuse `AiConversation(Message)` + `AiConversationRepository`; message execution goes through `AiProviderRegistry` + `CompliancePromptOrchestrator` (mock provider until `ai.feature_flags.enabled`). Conversation **metadata + token counts** are always persisted for this surface; message **content** only when `prompt_logging` is enabled.
- **Skills / Capabilities** read `QuenyxAiPlatform / QuenyxAiCapabilityCatalog` (Sprint 19) — no duplicated catalog.
- **Usage / Costs** are **derived** from real token counts; costs multiply by an optional `config('ai.workspace.pricing')` table and never fabricate currency.
- **Prompt Templates** (`ai_prompt_templates`), **Provider Settings** (`ai_provider_settings`, encrypted, write-only secrets), and **Permissions** (`ai_workspace_permissions`) add the missing governance concepts only.
- **Provider catalog & governance (RC1.1)**: `App\Services\Ai\AiProviderCatalog` declares the customer-visible provider catalog (OpenAI, Anthropic, Gemini, Azure OpenAI, OpenRouter, Mistral, Cohere, xAI Grok, Ollama, LM Studio, vLLM, LiteLLM, Hugging Face, Custom OpenAI-compatible). The catalog is **metadata only** — a provider is `executable` only when a real adapter exists in `config('ai.providers')` (today: **OpenAI**; plus the dev-only **mock**), and `platform_configured` only when real credentials are present. `AiProviderRegistry::defaultKey()` never returns `mock` in production: it prefers an explicit `AI_PROVIDER`, then OpenAI when configured, then `mock` only in local/testing, otherwise `''` (an honest "no provider configured" state). A real **test-connection** probe (`POST /api/ai/providers/{uuid}/test`) runs the adapter's `health()` for executable providers and reports `not_executable` for the rest — connectivity is never fabricated. Mock is excluded from the provider list outside local/testing.
- **Audit**: conversations, provider updates, template changes, and permission changes are logged via `AiAccessAuditLogger / AiWorkspaceAuditLogger` (never secrets or content).
- **Memory**: no durable AI memory store exists yet, so the Memory surface honestly reports "not enabled" instead of fabricating data.

Master switch: `ai.feature_flags.workspace_enabled` (env `AI_WORKSPACE_ENABLED`, default on; safe because chat falls back to the mock provider and metrics read 0 with no activity).